FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

LABEXER1

Vhinzenz Vielle Ellys B. Valencia	Doc. Hadji Tejuco
Name of Student	Name of Professor
10/16/2026	10/16/2026
Data Performed	Date Submitted

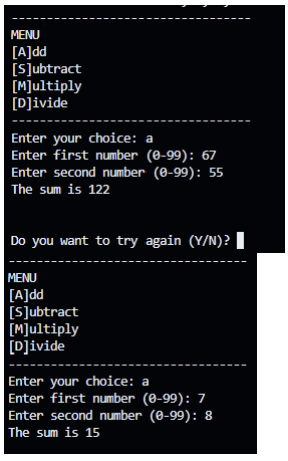
List the steps that you perform to come up with the program.

Program features
- Menu-driven addition, subtraction, multiplication, and division
- A two-digit number inputs (0–99)
- Supports negative subtraction results
- Displays quotient and remainder for division
- Rejects invalid inputs and division by zero
- Repeats the program when the user enters Y
- Clears the screen before returning to the menu
- Terminates when the user enters N
- Uses NASM procedures for input, output, validation, and number conversion

The program starts at _start, displays the menu, and reads the user's choice. Then it asks for two numbers using the read_number procedure, which validates the input and converts the typed characters into integers.
The chosen arithmetic operation is performed using Assembly instructions such as add, sub, imul, or div. The print_signed procedure converts the numerical result into characters for display.
Finally, the program asks whether to try again. Entering Y clears the terminal using ANSI escape codes and returns to the menu. Entering N exits the program.

Paste the source code and explain the functionalities and take a screenshot of the final output.

ADDITION:



```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: a
Enter first number (0-99): 91
Enter second number (0-99): 88
The sum is 179
```

SUBTRACTION:

```
Do you want to try again (Y/N)? y
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: s
Enter first number (0-99): 62
Enter second number (0-99): 99
The difference is -37


Do you want to try again (Y/N)?
```

```
---------------------------------
Enter your choice: s
Enter first number (0-99): 7
Enter second number (0-99): 8
The difference is -1


Do you want to try again (Y/N)?
```

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: s
Enter first number (0-99): 61
Enter second number (0-99): 43
The difference is 18


Do you want to try again (Y/N)?
```

MULTIPLICATION:

```
Do you want to try again (Y/N)? y
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: m
Enter first number (0-99): 15
Enter second number (0-99): 27
The product is 405


Do you want to try again (Y/N)?
```

```
---------------------------------
nter your choice: m
nter first number (0-99): 72
nter second number (0-99): 4
he product is 288


o you want to try again (Y/N)?
```

```
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: m
Enter first number (0-99): 99
Enter second number (0-99): 99
The product is 9801


Do you want to try again (Y/N)?
```

DIVIDE:

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: d
Enter first number (0-99): 27
Enter second number (0-99): 6
The quotient is 4 (remainder 3)
```

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: d
Enter first number (0-99): 81
Enter second number (0-99): 9
The quotient is 9 (remainder 0)
```

```
---------------------------------
Enter your choice: d
Enter first number (0-99): 67
Enter second number (0-99): 5
The quotient is 13 (remainder 2)
```

```
nasm -f elf32 calculator.asm -o calculator.o
ld -m elf_i386 calculator.o -o calculator
./calculator

; Build: nasm -f elf32 calculator.asm -o calculator.o
;        ld -m elf_i386 calculator.o -o calculator
; Run:   ./calculator

BITS 32
global _start

section .data
    menu db '---------------------------------',10
         db 'MENU',10
         db '[A]dd',10
         db '[S]ubtract',10
         db '[M]ultiply',10
         db '[D]ivide',10
         db '---------------------------------',10
```

```
    db 'Enter your choice: '

menu_len equ $ - menu
first_msg db 'Enter first number (0-99): '
first_len equ $ - first_msg

second_msg db 'Enter second number (0-99): '
second_len equ $ - second_msg
invalid_choice db 'Invalid choice. Select A, S, M, or D.',10
invalid_choice_len equ $ - invalid_choice

invalid_number db 'Invalid number. Enter a whole number from 0 to 99.',10
invalid_number_len equ $ - invalid_number

zero_msg db 'Cannot divide by zero. Please enter another second number.',10
zero_len equ $ - zero_msg

sum_msg db 'The sum is '
sum_len equ $ - sum_msg

sub_msg db 'The difference is '
sub_len equ $ - sub_msg

mul_msg db 'The product is '
mul_len equ $ - mul_msg

div_msg db 'The quotient is '
div_len equ $ - div_msg

rem_msg db ' (remainder '
rem_len equ $ - rem_msg
close_paren db ')'

again_msg db 10,10,'Do you want to try again (Y/N)? '
again_len equ $ - again_msg

invalid_again db 'Please enter Y or N.',10
invalid_again_len equ $ - invalid_again

bye_msg db 10,'Program terminated.',10
bye_len equ $ - bye_msg
lf db 10
```

```
    clear_seq db 27,'[2J',27,'[H'
    clear_len equ $ - clear_seq

section .bss
    input_buf resb 32
    number_buf resb 16
    choice resb 1
    first_num resd 1
    second_num resd 1
    result_num resd 1
    remainder_num resd 1
    char_buf resb 1

section .text
; write_bytes: ECX=address, EDX=byte count (clobbers EAX, EBX)
write_bytes:
    mov eax, 4
    mov ebx, 1
    int 0x80
    ret

; read_line: reads at most 31 chars, drains the remainder of long lines
; returns null-terminated input_buf, strips CR and LF; on EOF exits cleanly
read_line:
    push ebx
    push esi
    xor esi, esi
.read_character:
    mov eax, 3
    xor ebx, ebx
    mov ecx, char_buf
    mov edx, 1
    int 0x80
    cmp eax, 1
    jne .eof
    mov al, [char_buf]
    cmp al, 10
    je .complete
    cmp al, 13
    je .read_character
    cmp esi, 31
    jae .read_character
    mov [input_buf + esi], al
```

```
    inc esi
    jmp .read_character

.complete:
    mov byte [input_buf + esi], 0
    pop esi
    pop ebx
    ret

.eof:
    mov eax, 1
    xor ebx, ebx
    int 0x80

; read_number: prompt in ECX/EDX; returns EAX in [0,99]
; Re-prompts for empty / nondigit / too-large input.
read_number:
    push esi
    push edi
    mov esi, ecx
    mov edi, edx

.prompt:
    mov ecx, esi
    mov edx, edi
    call write_bytes
    call read_line
    mov ecx, input_buf
    xor eax, eax
    cmp byte [ecx], 0
    je .invalid

.parse:
    movzx edx, byte [ecx]
    test edx, edx
    jz .good
    cmp edx, '0'
    jb .invalid
    cmp edx, '9'
    ja .invalid
    sub edx, '0'
    imul eax, eax, 10
    add eax, edx
```

```
    cmp eax, 99
    ja .invalid
    inc ecx
    jmp .parse

.invalid:
    mov ecx, invalid_number
    mov edx, invalid_number_len
    call write_bytes
    jmp .prompt

.good:
    pop edi
    pop esi
    ret

; print_signed: signed EAX to standard output; supports negatives
print_signed:
    pushad
    mov edi, number_buf + 15
    xor ecx, ecx
    xor esi, esi
    test eax, eax
    jns .convert
    mov esi, 1
    neg eax

.convert:
    mov ebx, 10

.digit:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    inc ecx
    test eax, eax
    jnz .digit
    test esi, esi
    jz .emit
    dec edi
    mov byte [edi], '-'
```

```
    inc ecx

.emit:
    mov edx, ecx
    mov ecx, edi
    call write_bytes
    popad
    ret

_start:
.menu_loop:
    mov ecx, menu
    mov edx, menu_len
    call write_bytes
    call read_line
    mov al, [input_buf]
    cmp byte [input_buf + 1], 0
    jne .invalid_menu
    and al, 0DFh              ; ASCII lower-case -> upper-case
    cmp al, 'A'
    je .chosen
    cmp al, 'S'
    je .chosen
    cmp al, 'M'
    je .chosen
    cmp al, 'D'
    je .chosen

.invalid_menu:
    mov ecx, invalid_choice
    mov edx, invalid_choice_len
    call write_bytes
    jmp .menu_loop

.chosen:
    mov [choice], al
    mov ecx, first_msg
    mov edx, first_len
    call read_number
    mov [first_num], eax

.read_second:
    mov ecx, second_msg
```

```
    mov edx, second_len
    call read_number
    cmp byte [choice], 'D'
    jne .second_ok
    test eax, eax
    jnz .second_ok
    mov ecx, zero_msg
    mov edx, zero_len
    call write_bytes
    jmp .read_second

.second_ok:
    mov [second_num], eax
    mov eax, [first_num]
    mov ebx, [second_num]
    cmp byte [choice], 'A'
    je .add
    cmp byte [choice], 'S'
    je .subtract
    cmp byte [choice], 'M'
    je .multiply
    xor edx, edx
    div ebx
    mov [result_num], eax
    mov [remainder_num], edx
    mov ecx, div_msg
    mov edx, div_len
    jmp .show

.add:
    add eax, ebx
    mov [result_num], eax
    mov ecx, sum_msg
    mov edx, sum_len
    jmp .show

.subtract:
    sub eax, ebx
    mov [result_num], eax
    mov ecx, sub_msg
    mov edx, sub_len
    jmp .show
```

```
.multiply:
    imul eax, ebx
    mov [result_num], eax
    mov ecx, mul_msg
    mov edx, mul_len

.show:
    call write_bytes
    mov eax, [result_num]
    call print_signed
    cmp byte [choice], 'D'
    jne .end_result
    mov ecx, rem_msg
    mov edx, rem_len
    call write_bytes
    mov eax, [remainder_num]
    call print_signed
    mov ecx, close_paren
    mov edx, 1
    call write_bytes

.end_result:
    mov ecx, lf
    mov edx, 1
    call write_bytes

.ask_again:
    mov ecx, again_msg
    mov edx, again_len
    call write_bytes
    call read_line
    cmp byte [input_buf + 1], 0
    jne .invalid_again
    mov al, [input_buf]
    and al, 0DFh
    cmp al, 'Y'
    je .restart
    cmp al, 'N'
    je .quit

.invalid_again:
    mov ecx, invalid_again
    mov edx, invalid_again_len
```

```
    call write_bytes
    jmp .ask_again

.restart:
    mov ecx, clear_seq
    mov edx, clear_len
    call write_bytes
    jmp .menu_loop

.quit:
    mov ecx, bye_msg
    mov edx, bye_len
    call write_bytes
    mov eax, 1
    xor ebx, ebx
    int 0x80
```